VERİ ANALİSTİ MÜFREDATI 2026

MODÜL 5

SQL TEMEL

SELECT • WHERE • DISTINCT • ORDER BY • LIMIT • ALIAS • CASE WHEN

Modülün amacı
Bir veri tabanından doğru veriyi seçmek, filtrelemek, sıralamak ve iş sorularını SQL sorgularına çevirmek. Bu modülün sonunda basit ama gerçek analitik sorguları bağımsız yazabiliyor olmalısın.

Çıktı	Ne üreteceksin?
Analitik sorgular	E-ticaret veri tabanında temel iş sorularına cevap veren SQL sorguları
Mini case	Satış, müşteri, sipariş ve kategori bazlı temel analiz
Mülakat hazırlığı	Temel SQL mülakat soruları ve hata avı
Portfolyo adımı	README ile belgelenmiş SQL temel analiz dosyası

1. SQL neden veri analistinin temel aracıdır?

SQL (Structured Query Language), ilişkisel veritabanlarında saklanan veriyi sorgulamak için kullanılan dildir. Veri analistinin işi çoğu zaman “veriyi almak” ile başlar. Dashboard, Python analizi veya istatistiksel çalışma yapmadan önce ihtiyacın olan doğru kayıtları veritabanından çekebilmelisin.

- SQL ile milyonlarca satır içinden yalnızca gerekli kayıtları seçebilirsin.
- Filtreleme ve sıralamayı veritabanı tarafında yaparak gereksiz veri taşımayı önlersin.
- İş sorularını tekrar üretilebilir sorgulara dönüştürürsün.
- Analiz mantığın şeffaf olur; başka analistler sorgunu okuyup kontrol edebilir.

Analist bakış açısı
SQL yazarken amaç komut ezberlemek değil; iş sorusunu adım adım veri mantığına çevirmektir. “Hangi tablo?”, “hangi kolon?”, “hangi satırlar?”, “hangi sıralama?” soruları çoğu temel sorgunun iskeletini verir.

Örnek e-ticaret tablomuz: sales

order_id	order_date	customer	city	category	quantity	unit_price	total_amount	status
1001	2026-09-01	Ayşe	İstanbul	Electronics	1	32000	32000	Completed
1002	2026-09-01	Mehmet	Ankara	Home	2	4500	9000	Completed
1003	2026-09-02	Zeynep	İzmir	Electronics	1	18000	18000	Cancelled
1004	2026-09-03	Can	İstanbul	Sports	3	2200	6600	Completed
1005	2026-09-04	Ece	Bursa	Home	1	12500	12500	Completed

2. SELECT - hangi kolonları görmek istiyorsun?

SELECT bir sorgunun başlangıç noktasıdır. Hangi kolonların sonuç setinde gösterileceğini belirtir.

```
SELECT customer, total_amount
FROM sales;
```

Bu sorgu sales tablosundaki customer ve total_amount kolonlarını getirir.

SELECT * ne zaman kullanılır?

```
SELECT *
FROM sales;
```

Yıldız tüm kolonları getirir. Veri keşfinin ilk birkaç dakikasında faydalıdır; ancak üretim sorgularında genellikle yalnızca ihtiyaç duyduğun kolonları seçmek daha iyidir.

- Daha okunaklı sorgu
- Daha az veri transferi
- Şema değişikliklerinden daha az etkilenme
- Hangi alanların analize girdiğinin açık olması

Kolon sırası ve sorgu okunabilirliği

```
SELECT
    order_id,
    order_date,
    customer,
    total_amount
FROM sales;
```

Pratik kural
SELECT listesinde önce kimlik ve tarih, sonra boyutlar (müşteri, şehir, kategori), sonra ölçüler (quantity, amount) vermek analitik sorguları daha okunaklı yapar.

3. WHERE - hangi satırları istiyorsun?

WHERE, satır seviyesinde filtre uygular. Analistin en sık kullandığı koşul mekanizmasıdır.

Karşılaştırma operatörleri

Operatör	Anlam	Örnek
=	eşittir	city = 'İstanbul'
!= veya <>	eşit değildir	status <> 'Cancelled'
>	büyüktür	total_amount > 10000
>=	büyük veya eşit	quantity >= 2
<	küçüktür	unit_price < 5000
<=	küçük veya eşit	total_amount <= 20000

```
SELECT *
FROM sales
WHERE city = 'İstanbul';
```

AND / OR / NOT

```
SELECT customer, category, total_amount
FROM sales
WHERE city = 'İstanbul'
  AND total_amount > 10000;
```

```
SELECT *
FROM sales
WHERE city = 'İstanbul'
   OR city = 'Ankara';
```

Önemli
AND ve OR birlikte kullanıldığında parantez kullan. Operatör önceliğine güvenmek yerine niyetini açık yaz.

```
SELECT *
FROM sales
WHERE status = 'Completed'
  AND (city = 'İstanbul' OR city = 'Ankara');
```

4. WHERE ile sık kullanılan filtre kalıpları

IN

```
SELECT *
FROM sales
WHERE city IN ('İstanbul', 'Ankara', 'İzmir');
```

Aynı kolonda birden fazla olası değeri kontrol eder. Çok sayıda OR yazmaktan daha okunaklıdır.

BETWEEN

```
SELECT *
FROM sales
WHERE total_amount BETWEEN 10000 AND 30000;
```

BETWEEN çoğu SQL sisteminde alt ve üst sınırı dahil eder.

LIKE

```
SELECT *
FROM sales
WHERE customer LIKE 'A%';
```

'A%' A ile başlayan değerleri arar. '%tech%' içinde tech geçen değerleri bulur.

NULL kontrolü

```
SELECT *
FROM sales
WHERE city IS NULL;
```

```
SELECT *
FROM sales
WHERE city IS NOT NULL;
```

Kritik hata
NULL için = NULL yazma. NULL “bilinmeyen/eksik” anlamına gelir ve IS NULL / IS NOT NULL ile kontrol edilir.

5. DISTINCT - benzersiz değerleri görmek

DISTINCT, sonuç setindeki tekrar eden değerleri kaldırır. Veri keşfinde kategori, şehir, durum gibi alanları anlamak için çok kullanışlıdır.

```
SELECT DISTINCT city
FROM sales;

SELECT DISTINCT city, category
FROM sales;
```

İkinci sorgu şehir-kategori kombinasyonlarını benzersiz hale getirir; yalnızca city kolonunu değil.

Analitik kullanım örnekleri

- Veride kaç farklı durum etiketi kullanılmış?
- Şehir isimlerinde yazım varyasyonları var mı?
- Hangi ürün kategorileri mevcut?
- Bir tabloyu join etmeden önce anahtar alan gerçekten benzersiz mi?

DISTINCT her zaman çözüm değildir
JOIN sonrası duplicate oluştuysa SELECT DISTINCT ile semptomu gizlemek tehlikelidir. Önce tekrarın nedenini anlamalısın. Grain ve ilişki tipini kontrol et.

6. ORDER BY - sonucu anlamlı biçimde sıralamak

ORDER BY sorgu sonucunun sırasını belirler. ASC artan, DESC azalan sıralamadır. ASC varsayılandır.

```
SELECT customer, total_amount
FROM sales
ORDER BY total_amount DESC;
```

Birden fazla kolona göre sıralama

```
SELECT city, customer, total_amount
FROM sales
ORDER BY city ASC, total_amount DESC;
```

Önce şehir alfabetik sıralanır; aynı şehir içindeki kayıtlar total_amount değerine göre büyükten küçüğe sıralanır.

Top-N düşüncesinin temeli

“En yüksek tutarlı 10 sipariş” gibi sorular genellikle ORDER BY + LIMIT kombinasyonuyla çözülür.

7. LIMIT / TOP - kaç kayıt görmek istiyorsun?

LIMIT özellikle veri keşfi sırasında sonuç setini küçültmek için kullanılır. PostgreSQL, MySQL ve SQLite gibi sistemlerde yaygındır. SQL Server tarafında TOP kullanılır.

```
SELECT *
FROM sales
LIMIT 10;
```

En yüksek 5 sipariş

```
SELECT order_id, customer, total_amount
FROM sales
ORDER BY total_amount DESC
LIMIT 5;
```

SQL Server karşılığı

```
SELECT TOP 5 order_id, customer, total_amount
FROM sales
ORDER BY total_amount DESC;
```

Mantık
LIMIT tek başına “ilk 10” kaydı getirir; ama hangi 10 kaydın geleceğini anlamlı kılmak istiyorsan ORDER BY ile birlikte kullan.

8. Alias - sorguyu insan tarafından okunabilir yapmak

Alias, kolon veya tabloya geçici bir isim verir. AS anahtar kelimesi okunabilirliği artırır.

```
SELECT
    customer AS customer_name,
    total_amount AS revenue
FROM sales;
```

Hesaplanmış kolona alias

```
SELECT
    order_id,
    quantity * unit_price AS calculated_amount
FROM sales;
```

Alias özellikle dashboard veya rapor çıktısına gidecek sorgularda kolon isimlerini anlaşılır hale getirir.

Tablo alias mantığına hazırlık

```
SELECT s.customer, s.total_amount
FROM sales AS s;
```

JOIN konularında tablo alias kullanımı çok daha önemli hale gelir.

9. CASE WHEN - SQL içinde iş kuralı oluşturmak

CASE WHEN koşullu sınıflandırma yapar. Excel'deki IF mantığına benzer. Analistin ham veriyi iş anlamı taşıyan segmentlere çevirmesini sağlar.

```
SELECT
    order_id,
    total_amount,
    CASE
        WHEN total_amount >= 20000 THEN 'High Value'
        WHEN total_amount >= 10000 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS order_segment
FROM sales;
```

Duruma göre analitik etiket

```
SELECT
    order_id,
    status,
    CASE
        WHEN status = 'Cancelled' THEN 1
        ELSE 0
    END AS is_cancelled
FROM sales;
```

Sıra önemlidir
CASE WHEN koşulları yukarıdan aşağı değerlendirilir. Aralıkları mantıksal sırayla yaz; aksi halde bazı kayıtlar beklediğin segmente hiç ulaşmayabilir.

10. Temel SQL sorgusunun zihinsel modeli

Bir iş sorusunu sorguya çevirmek için aşağıdaki sırayı kullanabilirsin:

Adım	Soru	SQL parçası
1	Hangi veri kaynağı?	FROM
2	Hangi satırlar?	WHERE
3	Hangi kolonlar?	SELECT
4	Yeni iş etiketi gerekiyor mu?	CASE WHEN
5	Nasıl sıralanmalı?	ORDER BY
6	Kaç satır gerekli?	LIMIT

Örnek business question

“İstanbul ve Ankara’daki tamamlanmış siparişlerden 10.000 TL üstü olanları, en yüksek tutardan başlayarak göster. Siparişleri high/standard olarak etiketle.”

```
SELECT
    order_id,
    customer,
    city,
    total_amount,
    CASE
        WHEN total_amount >= 20000 THEN 'High'
        ELSE 'Standard'
    END AS value_segment
FROM sales
WHERE status = 'Completed'
  AND city IN ('İstanbul', 'Ankara')
  AND total_amount > 10000
ORDER BY total_amount DESC;
```

11. Gerçek analitik mini vaka: satış verisini keşfetme

Bir e-ticaret şirketinde yeni veri analisti olduğunu düşün. Sana sales tablosu verildi ve veri setini anlaman isteniyor.

Soru 1 - hangi kategoriler var?

```
SELECT DISTINCT category
FROM sales
ORDER BY category;
```

Soru 2 - iptal edilen siparişleri incele

```
SELECT order_id, order_date, customer, city, total_amount
FROM sales
WHERE status = 'Cancelled'
ORDER BY order_date DESC;
```

Soru 3 - yüksek değerli siparişleri bul

```
SELECT order_id, customer, category, total_amount
FROM sales
WHERE total_amount >= 20000
ORDER BY total_amount DESC;
```

Soru 4 - veri kalitesi kontrolü

```
SELECT *
FROM sales
WHERE customer IS NULL
   OR city IS NULL
   OR total_amount IS NULL;
```

Analiz alışkanlığı
SQL analizi sadece KPI hesaplamak değildir. İlk sorguların bir kısmı veri kalitesini, beklenmeyen kategorileri, NULL alanları ve sıra dışı değerleri anlamaya ayrılmalıdır.

12. Sık yapılan SQL hataları

Hata	Neden sorun?	Doğru yaklaşım
SELECT city='İstanbul' FROM sales	Filtre değil boolean ifade üretir	WHERE city = 'İstanbul' kullan
WHERE city = NULL	NULL eşitlik ile kıyaslanmaz	IS NULL kullan
LIMIT 10 ve ORDER BY yok	Hangi 10 satır olduğu analitik olarak belirsiz	İhtiyaca göre ORDER BY ekle
Her sorguda SELECT *	Gereksiz veri ve belirsiz bağımlılık	İhtiyaç duyulan kolonları yaz
JOIN duplicate sorununa DISTINCT basmak	Kök nedeni gizleyebilir	Grain ve join ilişkisini incele
CASE aralıklarını yanlış sırada yazmak	Kayıtlar yanlış sınıflanır	Koşulları üstten alta mantıksal sırala

SQL anahtar kelimelerinin yazım stili

SQL büyük/küçük harfe çoğu sistemde duyarsızdır; fakat SELECT, FROM, WHERE gibi anahtar kelimeleri büyük harfle yazmak sorguyu görsel olarak daha okunabilir yapar.

13. Mülakat tipi sorular

1. 1. SELECT ile SELECT DISTINCT arasındaki fark nedir?
2. 2. WHERE ve CASE WHEN hangi amaçlarla kullanılır?
3. 3. NULL bir değeri neden = NULL ile kontrol edemeyiz?
4. 4. WHERE status = 'Completed' AND city = 'İstanbul' OR city = 'Ankara' ifadesinde risk nedir?
5. 5. ORDER BY total_amount DESC LIMIT 5 sorgusu ne yapar?
6. 6. Bir kategoride yazım hatası olup olmadığını hızlıca nasıl kontrol edersin?
7. 7. SELECT * kullanımının üretim sorgularındaki dezavantajları nelerdir?
8. 8. CASE WHEN ile müşteri/sipariş segmentasyonu nasıl yapılır?
9. 9. DISTINCT ile duplicate problemini çözmek neden bazen yanlış yaklaşımdır?
10. 10. Bir business question'ı SQL'e çevirmeye hangi sorularla başlarsın?

Canlı SQL sorusu

sales tablosundan sadece tamamlanmış Electronics siparişlerini getir. 15.000 TL üstündekileri "Premium", diğerlerini "Standard" olarak etiketle. Sonucu en yüksek total_amount değerinden başlat ve ilk 20 satırı göster.

14. Uygulama alıştırmaları

1. sales tablosundaki tüm kayıtları getir.

2. Sadece customer, category ve total_amount kolonlarını getir.

3. İstanbul'daki siparişleri filtrele.

4. 10.000 TL'den yüksek siparişleri getir.

5. Completed olmayan siparişleri getir.

6. İstanbul, Ankara ve İzmir şehirlerini IN ile filtrele.

7. Benzersiz category listesini çıkar.

8. Siparişleri total_amount değerine göre büyükten küçüğe sırala.

9. En yüksek 10 siparişi getir.

10. customer kolonuna customer_name alias'ı ver.

11. CASE WHEN ile total_amount >= 20.000 ise High, >=10.000 ise Medium, diğerlerini Low yap.

12. NULL city veya customer içeren kayıtları veri kalitesi kontrolü için getir.

Çalışma yöntemi
Önce sorguyu kendin yaz. Sonra çalıştır, sonucu gözle kontrol et. Yanlışsa hata mesajını ezberlemek yerine SQL motorunun ne beklediğini anlamaya çalış.

15. Modül sonu portfolyo projesi

Proje adı: E-Commerce SQL Fundamentals Analysis

Amaç

Bir e-ticaret satış tablosunu temel SQL komutlarıyla keşfetmek ve iş sorularına cevap veren sorgular üretmek.

Teslim edilecekler

- 01_data_dictionary.md - kolon açıklamaları ve veri grain'i
- 02_exploration.sql - SELECT, DISTINCT ve NULL kontrolleri
- 03_filters.sql - WHERE, IN, BETWEEN, LIKE örnekleri
- 04_business_rules.sql - CASE WHEN segmentasyonları
- 05_top_orders.sql - ORDER BY + LIMIT analizleri
- README.md - iş soruları, sorguların amacı ve kısa bulgular

README'de anlatman gerekenler

- Dataset neyi temsil ediyor?
- Her satırın grain'i nedir?
- Hangi veri kalitesi problemlerini kontrol ettin?
- Hangi iş sorularını SQL'e çevirdin?
- Sorguların ürettiği 3-5 temel bulgu nedir?
- Sonraki modülde GROUP BY ve JOIN ile analizi nasıl genişleteceksin?

16. SQL Temel Cheat Sheet

Amaç	Kalıp
Kolon seç	SELECT col1, col2 FROM table;
Filtrele	SELECT * FROM table WHERE condition;
Çoklu değer	WHERE col IN ('A','B');
Aralık	WHERE amount BETWEEN 100 AND 500;
Metin eşleşmesi	WHERE name LIKE 'A%';
Eksik değer	WHERE col IS NULL;
Benzersiz değer	SELECT DISTINCT col FROM table;
Sırala	ORDER BY amount DESC;
Satır sınırla	LIMIT 10;
Alias	SELECT amount AS revenue FROM table;
Koşullu etiket	CASE WHEN condition THEN 'X' ELSE 'Y' END AS segment

Bu modülü bitirdiğinde
Temel bir tabloyu bağımsız keşfedebilmeli; ihtiyacın olan kolonları seçebilmeli, doğru kayıtları filtreleyebilmeli, sonuçları sıralayabilmeli ve CASE WHEN ile basit iş kuralları oluşturabilmelisin. Bir sonraki modül GROUP BY, aggregate fonksiyonlar, JOIN, HAVING, UNION ve subquery ile bu temelin üzerine kurulacak.